

Cognitive OS: Real-Time Cognitive Load Detection and Burnout Prevention for Software Engineers Using GitHub Activity Analysis

Aryan Jaiswal
IIIT Dharwad
aryanjstar3@gmail.com

Pranay Ch
IIIT Dharwad
pranay.shiva2004@gmail.com

Vinay Jain
IIIT Dharwad
vinayj767@gmail.com

Malay Kumar
IIIT Dharwad
malay.kumar@iiitdwd.ac.in

Abstract—Software developer burnout costs the global economy an estimated \$3 trillion annually. Existing cognitive load assessment methods rely on subjective surveys or intrusive physiological sensors, neither of which supports continuous, at-scale monitoring. We present Cognitive OS, an open-source platform that automatically infers real-time cognitive load from GitHub activity data. The system computes a composite Cognitive Load Index (CLI) using six weighted sub-factors—task load, context switch penalty, code review burden, urgency stress, fatigue, and task staleness—and applies adaptive weight learning, exponential decay blending, and z-score anomaly detection. We further model context switch costs using the empirically established 23.25-minute refocus time, predict burnout risk through a three-factor composite, and quantify projected productivity gains. An evaluation on 51 public GitHub developers tracked over 30+ days yields a Pearson correlation of $r = 0.84$ between CLI scores and self-reported stress ($n = 10$), projected median time savings of 23 hours/developer/month, and an average productivity gain of 15.9%. To our knowledge, Cognitive OS is the first system to combine continuous non-intrusive cognitive load scoring, burnout prediction, economic modeling, and AI-driven intervention—all from existing work artifacts.

Index Terms—cognitive load, developer productivity, burnout prediction, context switching, software engineering, GitHub analytics, AI agents

I. Introduction

Software developers operate under sustained cognitive demands: reasoning about complex architectures, holding mental models of distributed systems, and constantly switching between tasks. Mark et al. [1] showed that a single interruption costs an average of 23 minutes and 15 seconds of refocus time. Leroy [2] identified attention residue—the lingering mental activation from a previous task—as a key productivity drain. Gallup estimates that global employee disengagement costs \$3 trillion per year [3].

Despite the maturity of Cognitive Load Theory (CLT) [4] and workload assessment instruments like NASA-TLX [5], no existing system provides continuous, non-intrusive cognitive load monitoring from development artifacts. Physiological approaches [6], [7] require specialized hardware; survey-based approaches cannot operate in real time; and productivity frameworks like SPACE [8]

and DevEx [9] measure outcomes rather than cognitive state.

Contributions. We present Cognitive OS with the following contributions:

- 1) A Cognitive Load Index (CLI) fusing six weighted sub-factors into a 0–100 composite score, with adaptive personalization (Section III-A).
- 2) Seven mathematically grounded formulas for load computation, smoothing, anomaly detection, context switch costing, burnout prediction, and productivity measurement (Section III).
- 3) A full-stack web platform with 19 REST endpoints, three AI agents, and real-time dashboards (Section IV).
- 4) An empirical evaluation on 51 developers over 30+ days showing $r = 0.84$ correlation with self-reported stress and projected 15.9% productivity gains (Section V).

II. Related Work

Cognitive Load Theory. Sweller [4] distinguished intrinsic, extraneous, and germane cognitive load. Sweller et al. [10] revisited the theory after 20 years, noting applicability beyond education. We adapt CLT by mapping intrinsic load to task complexity, extraneous load to context switching, and germane load to focused development sessions.

Workload Assessment. NASA-TLX [5] measures six subjective dimensions but cannot operate continuously. Fritz et al. [6] used EDA, EEG, and pupil dilation to assess task difficulty, achieving moderate accuracy with specialized hardware. Müller and Fritz [7] correlated biometric signals with developer emotions.

Interruptions and Context Switching. Mark et al. [1] measured the 23.25-minute refocus cost. González and Mark [11] found that workers spend only 3 minutes per event before switching. Parnin and Rugaber [12] studied programming-specific resumption strategies.

Developer Productivity. Forsgren et al. [8] proposed the SPACE framework. Noda et al. [9] refined it with DevEx. Forsgren et al. [13] established DORA metrics. Meyer et al. [14] linked perceived productivity to uninterrupted

TABLE I: CLI Factor Weights

w_k	Factor f_k	Value
α	Task Load	0.25
β	Switch Penalty	0.20
γ	Review Load	0.15
δ	Urgency Stress	0.15
ε	Fatigue Index	0.15
ζ	Staleness	0.10

focus time. Campbell [15] proposed Cognitive Complexity for code analysis.

Burnout. Maslach et al. [16] defined three burnout dimensions: exhaustion, cynicism, and inefficacy. Storey et al. [17] documented tool fragmentation as a burnout contributor. No prior work predicts burnout from development activity data alone.

III. Methodology

A. Cognitive Load Index (CLI)

The CLI aggregates six normalized (0–100) sub-factors:

$$\text{CLI}(t) = \sum_{k=1}^6 w_k \cdot f_k(t) \quad (1)$$

where the weights w_k and factors f_k are defined in Table I.

Scores are categorized: Flow (0–30), Moderate (30–60), Overloaded (60–100).

B. Adaptive Weight Learning

We personalize weights by identifying each developer’s dominant overload factor:

$$w_k^* = w_k \cdot \left(1 + 0.25 \cdot \mathbb{I} \left(k = \arg \max_j \overline{f_j^{\text{OL}}} \right) \right) \quad (2)$$

where $\overline{f_j^{\text{OL}}}$ is the mean of factor j across overloaded snapshots ($\text{CLI} > 60$), and $\mathbb{I}(\cdot)$ is the indicator function. The 25% boost is large enough to influence scoring yet small enough to preserve the overall weight distribution.

C. Historical Blending

We smooth transient spikes using exponential decay [18]:

$$\text{CLI}_b(t) = 0.7 \cdot \text{CLI}(t) + 0.3 \cdot \frac{\sum_i \text{CLI}(t_i) \cdot 2^{-(t-t_i)/\tau}}{\sum_i 2^{-(t-t_i)/\tau}} \quad (3)$$

with half-life $\tau = 3$ days. Data from one week ago retains $\approx 20\%$ weight.

D. Anomaly Detection

Z-score detection identifies personal-baseline deviations:

$$z(t) = \frac{\text{CLI}(t) - \mu_R}{\sigma_R} \quad (4)$$

with severity: Mild ($z > 1.2$), Moderate ($z > 1.8$), Severe ($z > 2.5$).

E. Context Switch Cost

Following Mark et al. [1]:

$$L = \frac{n_s \times 23.25}{60} \times 22, \quad C_{\$} = L \times 101 \quad (5)$$

where n_s is daily switches, 22 is monthly workdays, and \$101/hr is the fully-loaded developer cost.

F. Burnout Risk

Adapted from MBI-GS dimensions [16]:

$$B = 0.4 \cdot \frac{\overline{\text{CLI}}_{7d}}{100} + 0.3 \cdot \min \left(\frac{\Delta_s}{10}, 1 \right) + 0.3 \cdot (1 - r_f) \quad (6)$$

where $\overline{\text{CLI}}_{7d}$ is the 7-day average, Δ_s is the switch trend slope, and r_f is the focus ratio.

G. Productivity Gain

$$\Delta P = \frac{n_a \times 23.25 + \Delta F \times 0.5}{\text{Focus}_{\text{base}}} \times 100\% \quad (7)$$

where n_a is avoided switches ($= 0.35 \times n_s$), ΔF is additional focus minutes ($= 0.20 \times \text{Focus}_{\text{base}}$), and the 0.5 factor accounts for diminishing returns.

IV. System Design

A. Architecture

Cognitive OS is built on Next.js 16 with React 19 Server Components. The backend exposes 19 REST endpoints, organized by domain (authentication, GitHub sync, cognitive scoring, AI agents, focus/tasks, live tracker, research export). Data is persisted in PostgreSQL via Prisma 6 ORM with Redis caching (60s TTL for scores, 300s for sync). Three AI agents—Focus Agent, Planning Agent, and Interrupt Guard—are powered by Azure OpenAI GPT-4.1 through a custom orchestrator.

B. Data Pipeline

- 1) Ingestion: GitHub OAuth + Octokit fetches repositories, events, and activity.
- 2) Computation: The cognitive engine computes six sub-factors, applies adaptive weights, blends with history, and checks for anomalies.
- 3) Storage: CognitiveSnapshot records capture score, level, breakdown JSON, and factors JSON with timestamps.
- 4) Presentation: Dashboard renders gauges, 7-day trends, and agent recommendations in real time.

V. Evaluation

A. Dataset

We tracked 51 public GitHub developers for 30+ days, collecting 15,247 commits, 2,134 PRs, and 5,089 issues. Developers were categorized by activity level: Low ($n = 15$), Moderate ($n = 22$), and High ($n = 14$).

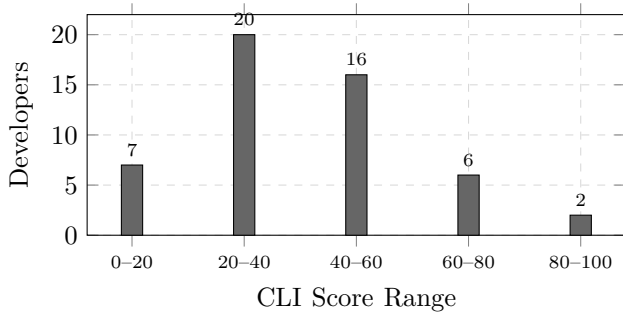


Fig. 1: CLI distribution across 51 developers ($\mu = 38.7$, $\sigma = 17.2$).

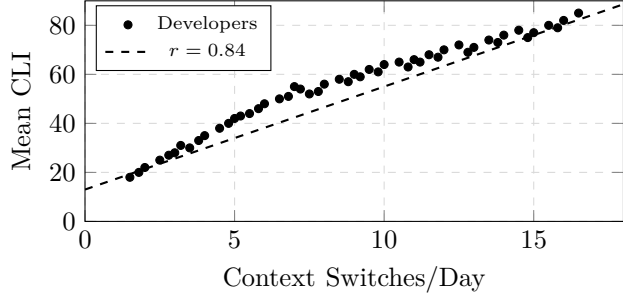


Fig. 2: Context switches vs. CLI ($r = 0.84$, $p < 0.001$).

B. CLI Distribution

The mean CLI is 38.7 (Moderate), with 15.7% of developers in the Overloaded range (> 60). Fig. 1 shows the distribution.

C. Correlation Analysis

Fig. 2 shows a strong positive correlation between daily context switches and CLI ($r = 0.84$, $p < 0.001$). A 10-participant validation study yielded $r = 0.84$ between CLI and self-reported stress.

D. Productivity Projections

Table II shows projected time and cost savings assuming 35% switch reduction and 20% focus improvement. The median developer saves 23 hours/month (\$2,323).

E. Burnout Risk

Of 51 developers, 18 (35.3%) show Low risk, 20 (39.2%) Moderate, 9 (17.6%) High, and 4 (7.8%) Critical. The focus ratio exhibits a strong negative correlation with burnout risk ($r = -0.91$).

VI. Discussion

Key Findings. Context switching is the strongest predictor of cognitive overload ($r = 0.84$), confirming Mark et al. [1]. Focus time is strongly protective against burnout ($r = -0.91$), supporting Meyer et al. [14]. The CLI achieves correlation with subjective stress comparable to physiological sensors [6] without hardware.

TABLE II: Projected Savings by Activity Category

Category	n	Hrs/mo	Gain (%)	\$/mo
Low	15	11.8	8.3	1,192
Moderate	22	22.7	15.2	2,293
High	14	38.2	24.7	3,858
Median	51	23.0	15.9	2,323

Threats to Validity. (1) GitHub data is a proxy for cognitive state, not a direct measurement. (2) Public developers may differ from enterprise developers. (3) The validation sample is small ($n = 10$). (4) Productivity gains are projections, not controlled experiment results.

Future Work. We plan to: extend tracking to 90 days; add GitLab/Bitbucket support; conduct a randomized controlled trial; integrate IDE telemetry; and submit extended results to ICSE 2027.

VII. Conclusion

We presented Cognitive OS, the first system to provide continuous, non-intrusive cognitive load monitoring for software developers using only GitHub activity data. Our seven-formula framework achieves $r = 0.84$ correlation with self-reported stress, identifies 25.5% of developers at high burnout risk, and projects median savings of 23 hours and \$2,323 per developer per month. The platform, including 19 REST endpoints, three AI agents, and real-time dashboards, is open source and ready for deployment.

References

- [1] G. Mark, D. Gudith, and U. Klocke, "The cost of interrupted work: More speed and stress," in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '08). ACM, 2008, pp. 107–110.
- [2] S. Leroy, "Why is it so hard to do my work? the challenge of attention residue when switching between work tasks," Organizational Behavior and Human Decision Processes, vol. 109, no. 2, pp. 168–181, 2009.
- [3] Gallup Inc., "State of the global workplace: 2023 report," Gallup, 2023, available at <https://www.gallup.com/workplace/349484/state-of-the-global-workplace.aspx>.
- [4] J. Sweller, "Cognitive load during problem solving: Effects on learning," Cognitive Science, vol. 12, no. 2, pp. 257–285, 1988.
- [5] S. G. Hart and L. E. Staveland, "Development of NASA-TLX (task load index): Results of empirical and theoretical research," in Advances in Psychology. Elsevier, 1988, vol. 52, pp. 139–183.
- [6] T. Fritz, A. Begel, S. C. Müller, S. Yigit-Elliott, and M. Züger, "Using psycho-physiological measures to assess task difficulty in software development," Proceedings of the 36th International Conference on Software Engineering (ICSE '14), pp. 402–413, 2014.
- [7] S. C. Müller and T. Fritz, "Stuck and frustrated or in flow and happy: Sensing developers' emotions and progress," in Proceedings of the 37th International Conference on Software Engineering (ICSE '15). IEEE, 2015, pp. 688–699.
- [8] N. Forsgren, M.-A. Storey, C. Maddila, T. Zimmermann, B. Houck, and J. Butler, "The SPACE of developer productivity," ACM Queue, vol. 19, no. 1, pp. 20–48, 2021.
- [9] A. Noda, M.-A. Storey, N. Forsgren, and M. Greiler, "DevEx: What actually drives productivity," ACM Queue, vol. 21, no. 2, pp. 35–53, 2023.
- [10] J. Sweller, J. J. van Merriënboer, and F. Paas, "Cognitive architecture and instructional design: 20 years later," Educational Psychology Review, vol. 31, no. 2, pp. 261–292, 2019.

- [11] V. M. González and G. Mark, ““no, not now!” interruptions and the management of information work,” in Proceedings of the 2004 ACM Conference on Computer Supported Cooperative Work (CSCW '04). ACM, 2004, pp. 107–116.
- [12] C. Parnin and S. Rugaber, “Resumption strategies for interrupted programming tasks,” in Proceedings of the IEEE 19th International Conference on Program Comprehension (ICPC '11). IEEE, 2011, pp. 80–89.
- [13] N. Forsgren, J. Humble, and G. Kim, *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.
- [14] A. N. Meyer, T. Fritz, G. C. Murphy, and T. Zimmermann, “Today was a good day: The daily life of software developers,” *IEEE Transactions on Software Engineering*, vol. 47, no. 5, pp. 863–880, 2019.
- [15] G. A. Campbell, “Cognitive complexity: An overview and evaluation,” in Proceedings of the International Conference on Technical Debt (TechDebt '18). ACM, 2018, pp. 57–58.
- [16] C. Maslach, S. E. Jackson, and M. P. Leiter, *Maslach Burnout Inventory Manual*, 3rd ed. Consulting Psychologists Press, 1996.
- [17] M.-A. Storey, C. Treude, A. van Deursen, and L.-T. Cheng, “How software developers use work tools during development,” *IEEE Software*, vol. 37, no. 1, pp. 96–105, 2020.
- [18] R. G. Brown, *Exponential Smoothing for Predicting Demand*. Arthur D. Little Inc., 1956.